

Revon Architecture

How the implemented platform stores versions, commits incrementally, and compares history without overstating the prototype.

The idea in one sentence

Revon is a content-addressed versioned key-value store whose state is indexed by a persistent fixed-depth Merkle hash trie. Normal mutation batches rewrite only touched paths, and Revon-H chooses between operation-log and hash-pruned Merkle differencing.

Implemented system



Layer	Responsibility	Current status
Frontend	Operates repositories and presents version, diff, storage, work, and integrity data.	Implemented for the local workflow.
HTTP API	Validates JSON/CSV requests, constrains repository paths, and maps structured errors.	Implemented; binds to loopback.
Revon-H	Creates immutable versions, copies affected trie paths, checks out history, and selects a diff path.	Implemented and tested.
Persistence	Stores addressed objects and advances version metadata and HEAD atomically.	Implemented with SQLite.
Evaluation	Runs deterministic baselines, forced-Merkle, Revon-H, and Dolt workloads with an oracle.	Audited evidence is versioned.

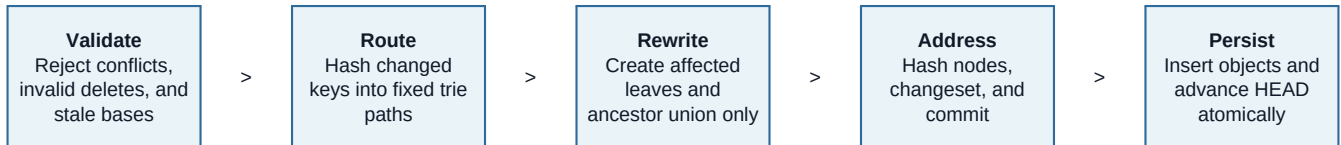
What exists now

- Durable repositories survive process exit and are verified when reopened.
- Each commit records a root hash, parent commit hash, addressed changeset, message, and timestamp.
- Unchanged trie nodes are reused by hash; normal mutations do not rebuild the complete state.
- Log, Merkle, and Hybrid diff modes return the same structured added, modified, and deleted result.

Incremental commit and durable storage

The data model remains content-addressed; SQLite supplies atomic durability.

Commit workflow



For branching factor **b**, depth **d**, and uniformly routed keys, expected leaf occupancy is approximately $N / (b^d)$. The paper configuration uses $b=8$ and $d=4$, yielding 4,096 possible leaf buckets. A sensitivity study reports other geometries instead of treating this choice as universal.

Stored objects and references

Record	Meaning
objects	Canonical immutable node, changeset, and commit payloads keyed by content hash.
versions	Human-readable version numbers mapped to addressed commit identities.
commit_stats	Changed-key, new/reused-node, byte, and timing measurements outside commit identity.
refs	Mutable named reference; the current implementation maintains HEAD.
metadata	Schema version, branching factor, depth, and Hybrid threshold.

Diff paths

Mode	Decision and work	Best fit
Log	Aggregate addressed changesets along an unbroken ancestor path; work follows intervening operations.	Nearby versions with short or sparse histories.
Merkle	Compare roots recursively, prune equal subtree hashes, and inspect entries only below unequal leaves.	Longer histories or cases where log aggregation becomes expensive.
Hybrid	Count ancestor operations and select Log at or below the configured boundary; select Merkle above it.	One policy over both exact algorithms.

The runtime default is 128 operations for demos and repositories created without an override. The audited paper profile uses a separately calibrated frozen boundary of 4,096 operations. Neither value is claimed to be hardware-independent.

What the project proves

The strongest explanation pairs measured behavior with explicit prototype boundaries.

Verified evidence

Evidence	Result
Final comparison	333 measured rows passed the shared correctness oracle; all 83 Tukey outlier candidates were retained.
Sensitivity	45 measured trials across five trie geometries were correct; b=8, d=4 minimized storage in the tested set.
Durability	Tests reopen SQLite repositories and reproduce history, checkout, diffs, metrics, and HEAD.
Failure handling	Injected transaction failure rolls back SQLite and restores the last durable in-memory HEAD.

Current boundaries

- History has one linear HEAD; named branches, merge commits, and conflict reporting are not implemented.
- Repositories allow one writer at a time; deployment-grade multi-host coordination is not implemented.
- Values remain inline in leaf nodes, and unreachable-object garbage collection is not implemented.
- The API is local-only and does not provide authentication, TLS, rate limiting, or public-service hardening.
- Dolt comparisons measure the declared CLI and SQL protocol, not isolated tree-algorithm speed.

A concise explanation

Revon stores each structured version behind a content-addressed root. A mutation batch rewrites the changed leaves and their shared ancestor paths, while unchanged subtrees keep their hashes. The new objects, changeset, commit, metrics, version alias, and HEAD are committed in one SQLite transaction. For comparison, Revon-H uses recorded operations for short histories and switches to hash-pruned traversal when the configured operation boundary is exceeded.

Reproduce locally

Run `python -m examples.revon_sqlite_demo --database revon_demo.revon.db --reset` for the durable reopen flow, and `python -m unittest discover -s tests -v` for the complete automated test suite.